

Taller Implementación de tácticas de disponibilidad: Ping/Echo y Redundancia Activa

1. Contexto del problema

RECAUDO-T es el servicio de consulta de saldo de la tarjeta del sistema de transporte masivo de la ciudad. Cada validador de bus consulta el saldo antes de autorizar el ingreso. La semana pasada, una falla de hardware dejó el servicio (una sola instancia) fuera de línea 11 minutos, y nadie se enteró hasta que los usuarios reclamaron. Le piden a su equipo que rediseñe el servicio para que (a) una caída se detecte automáticamente y (b) sea transparente para quien consulta.

Escenario de calidad de disponibilidad

Parte del escenario	Valor
Fuente del estímulo	Interna (un proceso servidor deja de responder)
Estímulo	Crash de una réplica
Respuesta esperada	El sistema detecta la falla, la registra, y sigue atendiendo con las réplicas restantes
Medida de la respuesta	Detección ≤ 3 s; el cliente no percibe errores

2. Marco conceptual

La disponibilidad se aproxima como:

$$A = \text{MTBF} / (\text{MTBF} + \text{MTTR})$$

Casi todas las tácticas de disponibilidad reducen el MTTR (tiempo de reparación), no el MTBF (qué tan seguido falla algo). Por eso siempre vienen en pareja detectar + reaccionar: no se puede reparar rápido algo que no se sabe que está roto.

- **Ping/Echo** (detectar): un monitor sondea periódicamente a cada componente y espera un eco dentro de un tiempo límite. Si no llega, asume que el componente falló. Es detección pura: no repara nada por sí sola.
- **Redundancia Activa** (recuperar, hot spare): varias réplicas atienden todas las solicitudes en paralelo; se usa la primera respuesta que llega. Al no depender de “levantar” nada, la recuperación toma milisegundos.

3. Cómo funciona el sistema

El dispatcher debe hacer, al mismo tiempo y sin que una tarea bloquee a la otra, dos cosas.

Por un lado, debe enviar un mensaje de sondeo (ping) a cada réplica cada T segundos. En caso de no obtener respuesta antes de t segundos, se considera un fallo en esa réplica. Si se acumulan k fallos consecutivos, el sistema debe marcarla como caída y dejar un registro del cambio. Si más adelante vuelve a responder m veces seguidas, el sistema debe marcarla como viva otra vez y dejar también el registro.

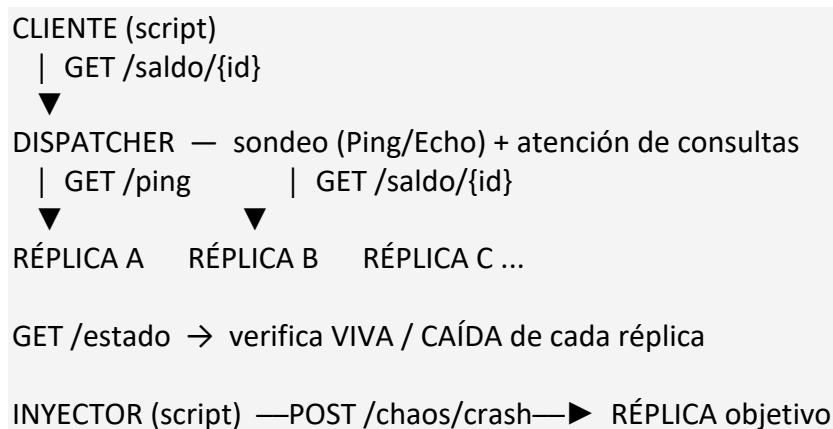
Por otro lado, cada vez que llegue una consulta de un cliente, el dispatcher debe enviarla al mismo tiempo a todas las réplicas que en ese momento estén marcadas como vivas, y responder al cliente con la primera respuesta que llegue, descartando las demás cuando lleguen.

Estas dos tareas (sondear y atender consultas) deben ejecutarse de forma independiente: atender una consulta no puede esperar a que termine un ciclo de sondeo, ni el sondeo debe detenerse mientras se atiende una consulta.

Por ejemplo: si una réplica se cae en el segundo 5, es probable que el dispatcher siga enviando consultas durante uno o dos segundos más, hasta que el sondeo detecte la falla. Pero como esas mismas consultas también se enviaron en paralelo a las otras réplicas, el cliente de todas formas recibe respuesta a tiempo y no nota que una réplica estaba caída. El sondeo, en ese momento, todavía no sabe nada; se entera un par de ciclos después, cuando acumula los k fallos seguidos y recién ahí queda registrado.

4. Arquitectura y contrato de cada componente

Aclaración: no hay hardware ni usuarios reales. Todo el sistema (incluido el “cliente”) es un conjunto de procesos de software que usted ejecuta en su máquina o en contenedores. El Cliente es un script que dispara solicitudes automáticamente a un ritmo constante, simulando múltiples validadores que preguntan a la vez; no es una interfaz gráfica. El Inyector de fallas es otro script corto que usted ejecuta a propósito para tumbar una réplica.



RÉPLICA

Proceso independiente, mínimo 2 (recomendado 3), cada una con su identificador (REPLICA_ID=A):

Ruta	Respuesta esperada
GET /ping	200 — {"replica_id":"A"} en menos de t
GET /saldo/{idTarjeta}	200 — {"replica_id":"A","saldo":15000}
POST /chaos/crash (solo la usa el inyector)	Termina el proceso de esa réplica

DISPATCHER

Único proceso, puerto fijo:

Ruta	Uso
GET /saldo/{idTarjeta}	La usa el cliente; reenvía en paralelo a las réplicas VIVA y responde con la primera válida (código 200 y cuerpo con saldo)
GET /estado	Devuelve {"A":"VIVA","B":"CAIDA",...} — le sirve para verificar en vivo que el monitor funciona

CLIENTE

Script que, durante DURACION segundos, a una tasa fija (sugerida: 20 solicitudes/s), llama a GET /saldo/1234 al dispatcher y, por cada intento, registra en un CSV: timestamp_envio, timestamp_respuesta, éxito (0/1), réplica.

INYECTOR

Script corto que usted ejecuta a mano: registra su propio timestamp y luego mata el proceso de la réplica elegida (docker kill, kill -9 o similar). El registro del timestamp es obligatorio: es su punto de referencia para calcular el tiempo de detección.

5. Requisitos a implementar

R1 Ping/Echo

- El monitor sondea GET /ping en todas las réplicas cada T segundos (sugerido: 1 s), sin bloquear la atención de las solicitudes.

- Tiempo de espera por sondeo t (sugerido: 300 ms); si transcurre sin respuesta, se considera un fallo.
- Tras k fallos consecutivos (sugerido 2), la réplica pasa a CAÍDA y se escribe en bitácora: [timestamp] REPLICA_B VIVA → CAÍDA.
- T , t y k deben ser configurables (a través de un archivo o variables de entorno), no constantes en el código.

R2 Redundancia Activa

- Cada consulta se envía en paralelo a todas las réplicas VIVA; se responde al cliente con la primera respuesta válida (200 + saldo disponible).
- El número de réplicas es configurable; agregar una no debe requerir modificar el código del dispatcher.
- GET /estado debe reflejar el estado real en todo momento.

Restricciones

1. La lógica se implementa, no se importa: no vale resolver esto con un balanceador con health check incorporado ni con una librería de resiliencia ya armada. Sí, puede usar librerías de red, de HTTP o de concurrencia.
2. Cada réplica es un proceso independiente (no un objeto dentro del mismo proceso).

6. Cómo medir: tiempo de detección y % de éxito

Con solo dos archivos tiene suficiente evidencia:

- **Bitácora del monitor** (del requisito R1): ahí está el timestamp del cambio de VIVA → CAÍDA.
- **CSV del cliente** (número 4): ahí están todas las solicitudes con sus respectivos resultados.

Además, registre el timestamp que el inyector escribe justo antes de matar la réplica (numeral 4) ese es el punto de partida.

Métrica	Cómo se calcula
Tiempo de detección	timestamp del cambio VIVA→CAÍDA en la bitácora – timestamp que registró el inyector. Si $T=1s$ y $k=2$, espere un valor cercano a 2 s.
% de solicitudes exitosas	filas con exito=1 / total de filas del CSV, durante todo el experimento.

Adjunte siempre el fragmento de log o el CSV del que proviene cada número. Un número sin evidencia no cuenta como medición.

7. Protocolo experimental

El cliente genera carga constante durante al menos 40 segundos; el inyector actúa a partir del segundo 15.

E0 Línea base. Sin fallas. Confirme en el CSV que más de una réplica responde (columna “réplica alternando”), así sabrá que la redundancia realmente está compitiendo entre réplicas y no favoreciendo siempre a una.

E1 Caída abrupta. El inyector mata una réplica. Verifique dos cosas, porque son evidencia de tácticas distintas: (a) en la bitácora y en GET /estado, que el monitor detectó la caída en un tiempo cercano al esperado, eso confirma Ping/Echo; (b) en el CSV del cliente, que no hay (o casi no hay) filas con exito=0 durante la ventana, eso confirma que la redundancia activa protegió al cliente. Ejecútelo dos veces para confirmar el tiempo de detección.

8. Entregables

1. Código fuente con instrucciones para levantarlo con un solo comando.
2. Bitácora del monitor y CSV del cliente de al menos una corrida de E1.
3. Tabla con el tiempo de detección y el % de éxito de E0 y E1.
4. Media página respondiendo las preguntas de análisis del numeral 10.

El sistema corre con al menos 2 réplicas en procesos separados; matar una réplica provoca un cambio de estado registrado en la bitácora en pocos segundos; el cliente no reporta errores (o casi ninguno) durante esa caída.

9. Preguntas de análisis

1. En E1, ¿el cliente dejó de recibir respuesta antes o después de que el monitor detectara la caída? ¿Por qué ocurre eso y qué le dice sobre la relación entre Ping/Echo y redundancia activa?
2. Si solo hubiera implementado Ping/Echo (sin redundancia), ¿el usuario habría notado la falla igual? ¿Y si solo hubiera implementado redundancia, sin monitor? ¿Para qué serviría entonces el monitor?

3. Si una réplica responde a /ping con normalidad pero devuelve un saldo incorrecto, ¿su monitor lo detectaría? ¿Por qué no, y qué táctica adicional del árbol de disponibilidad ayudaría en ese caso?
4. Suponga que cada réplica tiene una disponibilidad individual de 0,98 y que sus fallas son independientes entre sí. Calcule la disponibilidad del sistema con 2 y 3 réplicas. ¿En qué caso real ese supuesto de independencia sería falso?
5. El dispatcher es ahora el único punto por el que circula todo el tráfico. ¿Introdujo un nuevo punto único de falla al resolver el anterior?